

Imports

```
import qualified Data.Set as Set
```

Purpose: Imports the Data.Set module, which provides efficient set of data structures

```
import Data.Set (Set)
```

Purpose: Imports only the type constructor Set directly into your namespace.

```
import Data.List (sort, nub)
```

Purpose: Imports specific utility functions from the Data.List module.

Default type synonyms given.

```
type Pos = (Int, Int)

data Side = North | East | South | West deriving (Show, Eq, Ord)

type EdgePos = (Side, Int)

type Atoms = [Pos]

type Interactions = [(EdgePos, Marking)]
data Marking = Absorb | Reflect | Path EdgePos deriving (Show, Eq)
```

Challenge 1

```
getAdjacents :: Pos -> [Pos]
getAdjacents (c, r) = [(c-1, r), (c+1, r), (c, r-1), (c, r+1)]

add :: Pos -> (Int, Int) -> Pos
add (c, r) (dc, dr) = (c + dc, r + dr)

leftPerp, rightPerp :: (Int, Int) -> (Int, Int)
leftPerp (dc, dr) = (-dr, dc)
rightPerp (dc, dr) = (dr, -dc)
```

`getAdjacents`: Returns the four coordinates (North, South, East, West) immediately surrounding a given position (c, r).

`add`: Performs vector addition. Adds a direction vector (dc, dr) to a position (c, r) to calculate the next coordinate.

`leftPerp` / `rightPerp`: Takes a direction vector and calculates the vector pointing 90 degrees to the left or right, respectively. Used for changing direction during deflections.

```
getStartState :: Int -> EdgePos -> (Pos, (Int, Int))
getStartState n (North, c) = ((c, 0), (0, 1))
getStartState n (South, c) = ((c, n+1), (0, -1))
getStartState n (East, r) = ((n+1, r), (-1, 0))
getStartState n (West, r) = ((0, r), (1, 0))

isOutside :: Int -> Pos -> Bool
isOutside n (c,r) = c < 1 || c > n || r < 1 || r > n

posToEdgePos :: Int -> Pos -> EdgePos
posToEdgePos n (c,r)
  | r < 1      = (North, c)
  | r > n      = (South, c)
  | c < 1      = (West, r)
  | c > n      = (East, r)
  | otherwise = error $ "posToEdgePos: not an edge position " ++ show (c,r)
```

`getStartState`: Converts an `EdgePos` into a starting coordinate just outside the grid and an initial direction vector pointing inward.

`isOutside`: Returns True if a given coordinate is outside the boundaries of the NxN grid.

`posToEdgePos`: Converts a coordinate that has just exited the grid back into an `EdgePos` type (Side + Index)

```

getFrontDiagonals :: Pos -> (Int, Int) -> (Pos, Pos)
getFrontDiagonals pos dir =
  let lp = leftPerp dir
      rp = rightPerp dir
      frontLeft = add (add pos dir) lp
      frontRight = add (add pos dir) rp
  in (frontLeft, frontRight)

traceRay :: Int -> Set Pos -> EdgePos -> Pos -> (Int, Int) -> Set (Pos, (Int, Int)) -> Marking
traceRay n atomSet entryPos currentPos dir visited =
  let nextPos = add currentPos dir
      state = (currentPos, dir)
  in if isOutside n nextPos
    then let exitEdge = posToEdgePos n nextPos
         in if exitEdge == entryPos then Reflect else Path exitEdge
    else if Set.member nextPos atomSet
         then Absorb
         else
           if state `Set.member` visited then Reflect
           else
             let (fl, fr) = getFrontDiagonals currentPos dir
                 hasFL = Set.member fl atomSet
                 hasFR = Set.member fr atomSet
                 visited' = Set.insert state visited
             in case (hasFL, hasFR) of
                  (True, True)   -> Reflect
                  (True, False)  -> traceRay n atomSet entryPos currentPos (rightPerp dir) visited'
                  (False, True)  -> traceRay n atomSet entryPos currentPos (leftPerp dir) visited'
                  (False, False) -> traceRay n atomSet entryPos nextPos dir visited'

```

`getFrontDiagonals`: Calculates the two coordinates diagonally in front of the current position (front left and front right) relative to the current direction. Used to detect atoms that cause deflections.

`traceRay`: The recursive core of the simulation. It moves the ray one step at a time, checking for:

1. Exiting the grid (Result: Path or Reflect)
2. Hitting an atom directly (Result: Absorb)
3. Infinite loops (Result: Reflect)
4. Diagonal atoms causing 90-degree turns.

```

fireRay :: Int -> Set Pos -> EdgePos -> (EdgePos, Marking)
fireRay n atomSet entryPos =
  let
    (startPos, dir) = getStartState n entryPos

    firstStepPos = add startPos dir
  in
    if Set.member firstStepPos atomSet then
      (entryPos, Absorb)

    else
      let
        adjacentsToFirst = getAdjacents firstStepPos

        nextStepPos = add firstStepPos dir

        sideAtoms = filter (\pos -> pos /= nextStepPos) adjacentsToFirst

        isImmediateReflect = any (`Set.member` atomSet) sideAtoms
      in
        if isImmediateReflect then
          (entryPos, Reflect)

        else
          (entryPos, traceRay n atomSet entryPos startPos dir Set.empty)

calcBBInteractions :: Int -> Atoms -> Interactions
calcBBInteractions n atoms =
  let allEntries = map (North,) [1..n] ++ map (East,) [1..n] ++
    map (South,) [1..n] ++ map (West,) [1..n]
      atomSet = Set.fromList atoms
  in nub $ map (fireRay n atomSet) allEntries

```

fireRay: The wrapper for a single ray. It handles special “entry rules” before recursion starts:

1. Checks for immediate absorption (atom at entry)
2. Checks for immediate reflection (atom adjacent to entry)
3. Otherwise, delegates to traceRay.

calcBBInteractions: Generates all possible entry points for a grid of size N, fires a ray from each one, and returns the list of results.

Challenge 2

```
--challenge 2
combinations :: Int -> [a] -> [[a]]
combinations 0 _ = [[]]
combinations _ [] = []
combinations k (x:xs) = map (x:) (combinations (k-1) xs) ++ combinations k xs

inferGridSize :: Interactions -> Int
inferGridSize [] = 0
inferGridSize interactions =
  let indices = map \( (_, i), _ ) -> i interactions
  in maximum indices

verifyCandidate :: Int -> Set Pos -> Interactions -> Bool
verifyCandidate gridSize atomSet observedInteractions =
  all matches observedInteractions
  where
    matches (startPos, expectedMarking) =
      let (_, actualMarking) = fireRay gridSize atomSet startPos
      in actualMarking == expectedMarking

solveBB :: Int -> Int -> Interactions -> [Atoms]
solveBB sizeInput numAtoms interactions =
  let
    maxInteractionIndex = inferGridSize interactions
  in
    if maxInteractionIndex > sizeInput
    then error ("Input Error: Interaction index " ++ show maxInteractionIndex ++
      " is larger than grid size " ++ show sizeInput)
    else
      let
        gridSize = sizeInput
        allCoords = [(c, r) | c <- [1..gridSize], r <- [1..gridSize]]
        candidateAtoms = combinations numAtoms allCoords
      in
        filter (\atoms -> verifyCandidate gridSize (Set.fromList atoms) interactions) candidateAtoms
```

combinations: A combinatorial helper that generates all unique lists of length *k* from a given input list. Used to generate potential atom placements.

inferGridSize: Scans a list of interactions to find the largest index used, ensuring the grid is large enough to accommodate the input data.

verifyCandidate: Simulation function. Takes a candidate set of atoms and checks if firing rays through it produces exactly the same markings as the observed interactions.

solveBB: The main entry point for the solver. It accepts a user-defined grid size, the number of atoms to find, and the list of observed interactions.

1. **Validation:** It first checks if the provided grid size is valid; if any interaction uses a coordinate outside this size, it raises a clear error.

2. Generation: It calculates the search space by generating all unique combinations of atoms positions for the given grid size.
3. Filtering: It applies the verifyCandidate function to every combination, filtering the list to return only the atom configurations that perfectly reproduce the observed interactions.

Main

```
main :: IO ()
main = do
    print (calcBBInteractions 8 [(2,3),(7,3),(4,6),(7,8)])
    print (solveBB 8 4 [((North,1),Path (West,2)),((North,2),Absorb),((North,3),Path (North,6)),((North,4),Absorb),((North,5),Path
    print (solveBB 8 1 [((North,1), Absorb)])
    print (solveBB 8 1 [((North, 3), Absorb), ((West, 3), Absorb)])
    print (solveBB 8 1 [((North, 2), Path (West, 1))])
    print (solveBB 8 1 [((North, 1), Absorb), ((South, 1), Path (North, 1))])
    print (solveBB 4 1 [((North, 2), Absorb), ((West, 2), Absorb)])
    print (solveBB 10 1 [((North, 9), Absorb), ((West, 9), Absorb)])
    print (solveBB 3 1 [((North, 5), Absorb)])
```

How to run this code in ghci shell

Open text editor, paste the code into a new file and save the file as coursework2.hs

Save the code into the same folder as haskell and ghci.

Open terminal and type ghci.

Once able to see ghci as the prompt, type :load coursework2.hs

If successful will see

```
[ghci> :load coursework2.hs
[1 of 2] Compiling Main                ( coursework2.hs, interpreted )
Ok, one module loaded.
```

After this will be able to interact with the code.

Output screenshot

Challenge 1

```
[ghci> main
[((North,1),Path (West,2)),((North,2),Absorb),((North,3),Path (North,6)),((North
,4),Absorb),((North,5),Path (East,5)),((North,6),Path (North,3)),((North,7),Abso
rb),((North,8),Path (East,2)),((East,1),Path (West,1)),((East,2),Path (North,8))
,((East,3),Absorb),((East,4),Path (East,7)),((East,5),Path (North,5)),((East,6),
Absorb),((East,7),Path (East,4)),((East,8),Absorb),((South,1),Path (West,4)),((S
outh,2),Absorb),((South,3),Path (West,7)),((South,4),Absorb),((South,5),Path (We
st,5)),((South,6),Reflect),((South,7),Absorb),((South,8),Reflect),((West,1),Path
 (East,1)),((West,2),Path (North,1)),((West,3),Absorb),((West,4),Path (South,1))
,((West,5),Path (South,5)),((West,6),Absorb),((West,7),Path (South,3)),((West,8)
,Absorb)]
```

Challenge 2

```
[ghci> main
[[ (2,3), (4,6), (7,3), (7,8) ]]
[[ (1,1)], [(1,2)], [(1,3)], [(1,4)], [(1,5)], [(1,6)], [(1,7)], [(1,8)]]
[[ (3,3) ]]
[[ (3,2) ]]
[]
[[ (2,2) ]]
[[ (9,9) ]]
*** Exception: Input Error: Interaction index 5 is larger than grid size 3
CallStack (from HasCallStack):
  error, called at coursework2.hs:141:14 in main:Main
```